

CEL-Go Codelab：快速、安全、嵌入式運算式

來源：<https://codelabs.developers.google.com/codelabs/cel-go?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將使用以 Go 實作的一般運算語言編寫運算式。您將建立變數、使用邏輯和/或運算子、建構 JSON、建構 proto，以及調整運算式。

目錄

1. [1. 簡介](#)
2. [2. 核心概念](#)
3. [3. 設定](#)
4. [4. 向全世界問好！](#)
5. [5. 在函式中使用變數](#)
6. [6. 宣告變數](#)
7. [7. 邏輯 AND/OR](#)
8. [8. 自訂函式](#)
9. [9. 建構 JSON](#)
10. [10. 建構 Proto](#)
11. [11. 巨集](#)
12. [12. 調整](#)
13. [13. 涵蓋內容](#)

1. 簡介

什麼是 CEL ？

CEL 是一種非圖靈完備的運算式語言，設計宗旨是快速、可攜式且安全地執行。CEL 可單獨使用，也可嵌入較大的產品。

CEL 的設計宗旨是確保使用者程式碼執行安全無虞。盲目呼叫使用者 Python 程式碼中的 `eval()` 很危險，但您可以安全地執行使用者的 CEL 程式碼。此外，CEL 會避免導致效能降低的行為，因此評估時間安全無虞，僅需奈秒到微秒不等，非常適合用於效能至關重要的應用程式。

CEL 會評估運算式，這類運算式類似於單行函式或 `lambda` 運算式。雖然 CEL 通常用於布林決策，但也可以用來建構更複雜的物件，例如 JSON 或 protobuf 訊息。

CEL 適合您的專案嗎？

由於 CEL 會在奈秒到微秒內評估 AST 中的運算式，因此 CEL 的理想用途是效能關鍵路徑的應用程式。不應在重要路徑中將 CEL 程式碼編譯為 AST；理想的應用程式是經常執行設定，但修改頻率相對較低的應用程式。

舉例來說，針對服務的每個 HTTP 要求執行安全性政策，就是 CEL 的理想用途，因為安全性政策很少變更，而且 CEL 對回應時間的影響微乎其微。在本例中，CEL 會傳回布林值，指出是否應允許要求，但也可以傳回更複雜的訊息。

本程式碼研究室涵蓋哪些內容？

本程式碼研究室的第一個步驟會說明使用 CEL 的動機，以及 **核心概念**。其餘部分則專門介紹涵蓋常見用途的程式碼練習。如要深入瞭解語言、語意和功能，請參閱 GitHub 上的 [CEL 語言定義](#) 和 [CEL Go 文件](#)。

本程式碼研究室適用於想學習 CEL，以便使用已支援 CEL 的服務的開發人員。本程式碼研究室不會說明如何將 CEL 整合至自己的專案。

課程內容

- CEL 的核心概念
- Hello, World：使用 CEL 評估字串
- 建立變數
- 瞭解 CEL 在邏輯 AND/OR 運算中的短路行為
- 如何使用 CEL 建構 JSON
- 如何使用 CEL 建構 Protobuffer
- 建立巨集
- 如何調整 CEL 運算式

軟硬體需求

- [Go](#)

必要條件

本程式碼研究室以 [Protocol Buffers](#) 和 [Go Lang](#) 的基本概念為基礎。

如果您不熟悉 Protocol Buffers，第一個練習會讓您瞭解 CEL 的運作方式，但由於進階範例會使用 Protocol Buffers 做為 CEL 的輸入內容，因此可能較難理解。建議先完成其中一項[教學課程](#)。請注意，使用 CEL 時不一定要使用 Protocol Buffers，但本程式碼研究室會大量使用這項工具。

如要測試是否已安裝 Go，請執行下列指令：



go --help

2. 核心概念

應用程式

CEL 適用於一般用途，已用於各種應用程式，從 RPC 路由到定義安全政策皆可。CEL 具有可擴充性，與應用程式無關，且經過最佳化調整，適合編譯一次、評估多次的工作流程。

許多服務和應用程式都會評估宣告式設定。舉例來說，角色型存取控管 (RBAC) 是一種宣告式設定，可根據角色和一組使用者產生存取決策。如果宣告式設定是 80% 的使用案例，那麼當使用者需要更強大的表達能力時，CEL 就是補足其餘 20% 的實用工具。

編譯

運算式是針對環境編譯而成。編譯步驟會產生 protobuf 形式的抽象語法樹 (AST)。編譯後的運算式通常會儲存起來，以供日後使用，盡可能加快評估速度。單一編譯運算式可使用許多不同的輸入內容進行評估。

運算式

使用者定義運算式，服務和應用程式則定義運算式的執行環境。函式簽章會宣告輸入內容，並寫在 CEL 運算式之外。系統會自動匯入 CEL 可用的函式庫。

在以下範例中，運算式會採用要求物件，而要求包含聲明權杖。運算式會傳回布林值，指出聲明權杖是否仍有效。

```
// Check whether a JSON Web Token has expired by inspecting the 'exp' claim.
//
// Args:
//   claims - authentication claims.
//   now    - timestamp indicating the current system time.
// Returns: true if the token has expired.
//
timestamp(claims["exp"]) < now
```

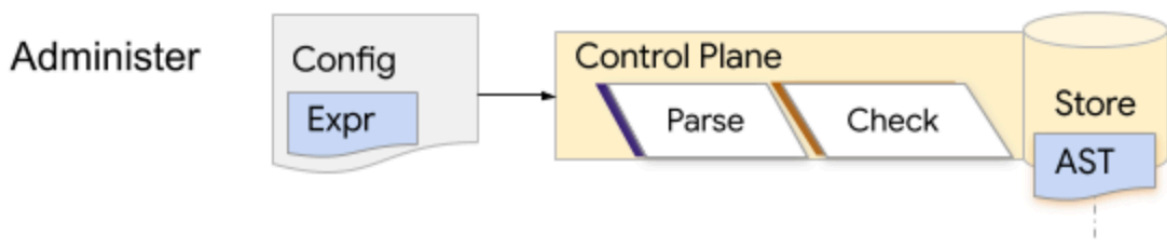
環境

環境是由服務定義。內嵌 CEL 的服務和應用程式會宣告運算式環境。環境是可在運算式中使用的變數和函式集合。

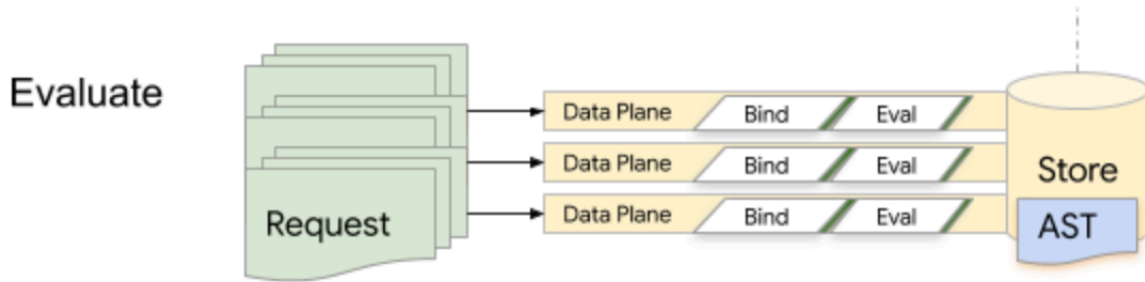
CEL 型別檢查程式會使用以 Proto 為基礎的宣告，確保運算式中的所有 ID 和函式參照都已正確宣告及使用。

剖析運算式的三個階段

運算式處理程序分為三個階段：剖析、檢查和評估。CEL 最常見的模式是控制平面在設定時剖析及檢查運算式，並儲存 AST。



在執行階段，資料層會重複擷取及評估 AST。CEL 經過最佳化，可提升執行階段效率，但不應在延遲時間至關重要的程式碼路徑中進行剖析和檢查。



警告：不應在延遲時間至關重要的程式碼路徑中進行剖析和檢查。

系統會使用 [ANTLR](#) 詞法分析器 / 剖析器文法，將人類可讀的運算式剖析為抽象語法樹狀結構。剖析階段會發出以 Proto 為基礎的抽象語法樹狀結構，其中 AST 中的每個 Expr 節點都包含整數 ID，用於建立剖析和檢查期間產生的中繼資料索引。剖析期間產生的 [syntax.proto](#) 會忠實呈現以運算式字串形式輸入的內容抽象表示法。

剖析運算式後，系統可能會根據環境檢查運算式，確保運算式中的所有變數和函式 ID 都已宣告，且使用方式正確。型別檢查程式會產生 [checked.proto](#)，其中包含型別、變數和函式解析中繼資料，可大幅提升評估效率。

最佳做法：執行型別檢查，即使是 JSON 等型別推斷受限的動態資料，也能提升剖析運算式的速度和安全性。

CEL 評估工具需要 3 項資訊：

- 任何自訂擴充功能的函式繫結
- 變數繫結
- 要評估的 AST

函式和變數繫結應與用於編譯 AST 的繫結相符。這些輸入內容都可以在多項評估中重複使用，例如針對多組變數繫結評估的 AST，或針對多個 AST 使用的相同變數，或是在程序生命週期內使用的函式繫結 (常見情況)。

步驟 3 · 約 2 分鐘

3. 設定

本程式碼研究室的程式碼位於 `cel-go` 存放區的 `codelab` 資料夾中。解決方案位於相同存放區的 `codelab/solution` 資料夾中。

複製並 `cd` 到存放區：

```
git clone https://github.com/google/cel-go.git
cd cel-go/codelab
```

使用 `go run` 執行程式碼：

```
go run .
```

您應該會看到以下的輸出內容：

```
=== Exercise 1: Hello World ===

=== Exercise 2: Variables ===

=== Exercise 3: Logical AND/OR ===

=== Exercise 4: Customization ===

=== Exercise 5: Building JSON ===

=== Exercise 6: Building Protos ===

=== Exercise 7: Macros ===

=== Exercise 8: Tuning ===
```

CEL 封裝在哪裡？

在編輯器中開啟 `codelab/codelab.go`。您應該會看到主函式，該函式會驅動本程式碼研究室中練習的執行作業，後面接著三組輔助函式。第一組輔助程式可協助進行 CEL 評估階段：

- `Compile` 函式：根據環境剖析及檢查輸入運算式
- `Eval` 函式：根據輸入內容評估已編譯的程式
- `Report` 函式：以易讀格式列印評估結果

此外，我們也提供 `request` 和 `auth` 輔助程式，協助您建構各種練習的輸入內容。

練習會使用套件簡短名稱參照套件。如要深入瞭解詳細資料，請參閱 `google/cel-go` 存放區中從套件到來源位置的對應：

套件	來源位置	說明
----	------	----

cel	cel-go/cel	頂層介面
ref	cel-go/common/types/ref	參考介面
類型	cel-go/common/types	執行階段類型值

步驟 4 · 約 5 分鐘

4. 向全世界問好！

按照所有程式設計語言的慣例，我們將從建立及評估「Hello World!」開始。

設定環境

在編輯器中找出 `exercise1` 的宣告，然後填入下列內容來設定環境：

```
// exercise1 evaluates a simple literal expression: "Hello, World!"
//
// Compile, eval, profit!
func exercise1() {
    fmt.Println("=== Exercise 1: Hello World ===\n")
    // Create the standard environment.
    env, err := cel.NewEnv()
    if err != nil {
        glog.Exitf("env error: %v", err)
    }
    // Will add the parse and check steps here
}
```

CEL 應用程式會根據環境評估運算式。 `env, err := cel.NewEnv()` 會設定標準環境。

您可以為呼叫提供 `cel.EnvOption` 選項，自訂環境。這些選項可停用巨集、宣告自訂變數和函式等。

標準 CEL 環境支援[語言規格](#)中定義的所有型別、運算子、函式和巨集。

剖析及檢查運算式

環境設定完成後，即可剖析及檢查運算式。在函式中新增下列內容：

```
// exercisel evaluates a simple literal expression: "Hello, World!"
//
// Compile, eval, profit!
func exercisel() {
    fmt.Println("=== Exercise 1: Hello World ===\n")
    // Create the standard environment.
    env, err := cel.NewEnv()
    if err != nil {
        glog.Exitf("env error: %v", err)
    }
    // Check that the expression compiles and returns a String.
    ast, iss := env.Parse(`"Hello, World!"`)
    // Report syntactic errors, if present.
    if iss.Err() != nil {
        glog.Exit(iss.Err())
    }
    // Type-check the expression for correctness.
    checked, iss := env.Check(ast)
    // Report semantic errors, if present.
    if iss.Err() != nil {
        glog.Exit(iss.Err())
    }
    // Check the output type is a string.
    if !reflect.DeepEqual(checked.OutputType(), cel.StringType) {
        glog.Exitf(
            "Got %v, wanted %v output type",
            checked.OutputType(), cel.StringType,
        )
    }
    // Will add the planning step here
}
```

`Parse` 和 `Check` 呼叫傳回的 `iss` 值是問題清單，可能包含錯誤。如果 `iss.Err()` 為非空值，表示語法或語意有誤，程式無法繼續執行。如果運算式格式正確，這些呼叫的結果就是可執行的 `cel.Ast`。

注意：在本程式碼研究室中，剖析和檢查階段已併入 `compile()` 輔助程式，可用於其餘練習。

評估運算式

運算式剖析並檢查到 `cel.Ast` 後，即可轉換為可評估的程式，其函式繫結和評估模式可透過函式選項自訂。請注意，您也可以使用 `cel.CheckedExprToAst` 或 `cel.ParsedExprToAst` 函式，從 Proto 讀取 `cel.Ast`。

規劃 `cel.Program` 後，即可呼叫 `Eval`，根據輸入內容評估 `cel.Program`。`Eval` 的結果會包含結果、評估詳細資料和錯誤狀態。

新增規劃並呼叫 `eval`：

```
// exercisel evaluates a simple literal expression: "Hello, World!"
//
// Compile, eval, profit!
func exercisel() {
    fmt.Println("=== Exercise 1: Hello World ===\n")
    // Create the standard environment.
    env, err := cel.NewEnv()
    if err != nil {
        glog.Exitf("env error: %v", err)
    }
    // Check that the expression compiles and returns a String.
    ast, iss := env.Parse(`"Hello, World!"`)
    // Report syntactic errors, if present.
    if iss.Err() != nil {
        glog.Exit(iss.Err())
    }
    // Type-check the expression for correctness.
    checked, iss := env.Check(ast)
    // Report semantic errors, if present.
    if iss.Err() != nil {
        glog.Exit(iss.Err())
    }
    // Check the output type is a string.
    if !reflect.DeepEqual(checked.OutputType(), cel.StringType) {
        glog.Exitf(
            "Got %v, wanted %v output type",
            checked.OutputType(), cel.StringType)
    }
    // Plan the program.
    program, err := env.Program(checked)
    if err != nil {
        glog.Exitf("program error: %v", err)
    }
    // Evaluate the program without any additional arguments.
    eval(program, cel.NoVars())
    fmt.Println()
}
```

為簡潔起見，我們會在日後的練習中省略上述錯誤案例。

執行程式碼

在指令列中重新執行程式碼：

```
go run .
```

您應該會看到下列輸出內容，以及未來練習的預留位置。

```
=== Exercise 1: Hello World ===
```

```
----- input -----
```

```
(interpreter.emptyActivation)
```

```
----- result -----
```

```
value: Hello, World! (types.String)
```

5. 在函式中使用變數

大多數 CEL 應用程式都會宣告可在運算式中參照的變數。變數宣告會指定名稱和型別。變數的型別可以是 [CEL 內建型別](#)、通訊協定緩衝區的已知型別，或是任何 protobuf 訊息型別，只要其描述元也提供給 CEL 即可。

注意：在下列範例中，`compile` 和 `eval` 輔助程式已納入為 `env.Parse()` / `env.Check()` 和 `program.Eval()` 的替代項目。這些輔助函式用於列印練習的輸入和輸出內容。

新增函式

在編輯器中找出 `exercise2` 的宣告，然後新增下列內容：

```
// exercise2 shows how to declare and use variables in expressions.
//
// Given a request of type google.rpc.context.AttributeContext.Request
// determine whether a specific auth claim is set.
func exercise2() {
    fmt.Println("=== Exercise 2: Variables ===\n")
    env, err := cel.NewEnv(
        // Add cel.EnvOptions values here.
    )
    if err != nil {
        glog.Exit(err)
    }
    ast := compile(env, `request.auth.claims.group == 'admin'`, cel.BoolType)
    program, _ := env.Program(ast)

    // Evaluate a request object that sets the proper group claim.
    claims := map[string]string{"group": "admin"}
    eval(program, request(auth("user:me@acme.co", claims), time.Now()))
    fmt.Println()
}
```

重新執行並瞭解錯誤

重新執行程式：

```
go run .
```

您應該會看到以下的輸出內容：

```
ERROR: <input>:1:1: undeclared reference to 'request' (in container '')
| request.auth.claims.group == 'admin'
| ^
```

型別檢查器會產生要求物件的錯誤，方便您查看發生錯誤的來源程式碼片段。

步驟 6 · 約 5 分鐘

6. 宣告變數

新增「`EnvOptions`」

在編輯器中，請為要求物件提供 `google.rpc.context.AttributeContext.Request` 類型的訊息宣告，修正產生的錯誤，如下所示：

```
// exercise2 shows how to declare and use variables in expressions.
//
// Given a `request` of type `google.rpc.context.AttributeContext.Request`
// determine whether a specific auth claim is set.
func exercise2() {
    fmt.Println("=== Exercise 2: Variables ===\n")
    env, err := cel.NewEnv(
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
    )
    if err != nil {
        glog.Exit(err)
    }
    ast := compile(env, `request.auth.claims.group == 'admin'`, cel.BoolType)
    program, _ := env.Program(ast)

    // Evaluate a request object that sets the proper group claim.
    claims := map[string]string{"group": "admin"}
    eval(program, request(auth("user:me@acme.co", claims), time.Now()))
    fmt.Println()
}
```

重新執行並瞭解錯誤

再次執行程式：

```
go run .
```

您應該會看到下列錯誤訊息：

```
ERROR: <input>:1:8: [internal] unexpected failed resolution of
'google.rpc.context.AttributeContext.Request'
| request.auth.claims.group == 'admin'
| .....^
```

如果要使用參照 protobuf 訊息的變數，型別檢查程式也需要知道型別描述元。

附註：[型別描述元](#)是 protobuf 訊息，用於描述 protobuf 訊息的欄位宣告。描述元可以視為反映型別。型別檢查器會使用描述元判斷欄位型別參照。描述元可以儲存，也可以從 protobuf 訊息例項讀取。`cel.Types()` 輔助程式會檢查訊息例項，判斷類型檢查器可用的描述元集。`cel.EnvOption`

使用 `cel.Types` 判斷函式中要求的描述元：

```
// exercise2 shows how to declare and use variables in expressions.
//
// Given a `request` of type `google.rpc.context.AttributeContext.Request`
// determine whether a specific auth claim is set.
func exercise2() {
    fmt.Println("=== Exercise 2: Variables ===\n")
    env, err := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request")),
    ),
)

    if err != nil {
        glog.Exit(err)
    }
    ast := compile(env, `request.auth.claims.group == 'admin'`, cel.BoolType)
    program, _ := env.Program(ast)

    // Evaluate a request object that sets the proper group claim.
    claims := map[string]string{"group": "admin"}
    eval(program, request(auth("user:me@acme.co", claims), time.Now()))
    fmt.Println()
}
```

已成功重新執行！

再次執行程式：

```
go run .
```

畫面應顯示如下：


```
=== Exercise 2: Variables ===

request.auth.claims.group == 'admin'

----- input -----
request = time: <
  seconds: 1569255569
>
auth: <
  principal: "user:me@acme.co"
  claims: <
    fields: <
      key: "group"
      value: <
        string_value: "admin"
      >
    >
  >
>

----- result -----
value: true (types.Bool)
```

回顧一下，我們剛才宣告了錯誤的變數，並為其指派型別描述元，然後在運算式評估中參照該變數。

7. 邏輯 AND/OR

CEL 的獨特功能之一是使用可交換的邏輯運算子。即使發生錯誤或輸入不完整，條件分支的任一側都可以中斷評估。

換句話說，CEL 會盡可能找出可產生結果的評估順序，並忽略其他評估順序中可能發生的錯誤，甚至是缺少的資料。應用程式可依據這項屬性盡量減少評估成本，在沒有昂貴的輸入內容也能得出結果時，延後收集這些內容。

我們會新增 AND/OR 範例，然後嘗試使用不同輸入內容，瞭解 CEL 如何短路評估。

建立函式

在編輯器中，將以下內容新增至練習 3：

```
// exercise3 demonstrates how CEL's commutative logical operators work.
//
// Construct an expression which checks if the `request.auth.claims.group`
// value is equal to admin or the `request.auth.principal` is
// `user:me@acme.co`. Issue two requests, one that specifies the proper
// user, and one that specifies an unexpected user.
func exercise3() {
    fmt.Println("=== Exercise 3: Logical AND/OR ===\n")
    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
    )
}
```

接著，加入這個 OR 陳述式，如果使用者是 `admin` 群組的成員，或具有特定電子郵件 ID，則會傳回 `true`：

```
// exercise3 demonstrates how CEL's commutative logical operators work.
//
// Construct an expression which checks if the `request.auth.claims.group`
// value is equal to admin or the `request.auth.principal` is
// `user:me@acme.co`. Issue two requests, one that specifies the proper
// user, and one that specifies an unexpected user.
func exercise3() {
    fmt.Println("=== Exercise 3: Logical AND/OR ===\n")
    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
    )

    ast := compile(env,
        `request.auth.claims.group == 'admin'
        || request.auth.principal == 'user:me@acme.co'`,
        cel.BoolType)
    program, _ := env.Program(ast)
}
```

最後，新增 `eval` 案例，使用空白聲明集評估使用者：

```
// exercise3 demonstrates how CEL's commutative logical operators work.
//
// Construct an expression which checks whether the request.auth.claims.group
// value is equal to admin or the request.auth.principal is
// user:me@acme.co. Issue two requests, one that specifies the proper user,
// and one that specifies an unexpected user.
func exercise3() {
    fmt.Println("=== Exercise 3: Logical AND/OR ===\n")
    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
    )

    ast := compile(env,
        `request.auth.claims.group == 'admin'
        || request.auth.principal == 'user:me@acme.co'`,
        cel.BoolType)
    program, _ := env.Program(ast)

    emptyClaims := map[string]string{}
    eval(program, request(auth("user:me@acme.co", emptyClaims), time.Now()))
}
```

專業提示：請務必閱讀這個檔案中的 `request()`、`auth()` 和輔助函式。以及 `time.Now()`，有助於調整評估的輸入內容。

使用空白聲明集執行程式碼

重新執行程式，您應該會看到下列新輸出內容：

```
=== Exercise 3: Logical AND/OR ===

request.auth.claims.group == 'admin'
  || request.auth.principal == 'user:me@acme.co'

----- input -----
request = time: <
  seconds: 1569302377
>
auth: <
  principal: "user:me@acme.co"
  claims: <
    >
  >
>

----- result -----
value: true (types.Bool)
```

更新評估案件

接著，請更新評估案例，傳遞具有空白聲明集的不同主體：

```
// exercise3 demonstrates how CEL's commutative logical operators work.
//
// Construct an expression which checks whether the request.auth.claims.group
// value is equal to admin or the request.auth.principal is
// user:me@acme.co. Issue two requests, one that specifies the proper user,
// and one that specifies an unexpected user.
func exercise3() {
    fmt.Println("=== Exercise 3: Logical AND/OR ===\n")
    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
    )

    ast := compile(env,
        `request.auth.claims.group == 'admin'
        || request.auth.principal == 'user:me@acme.co'`,
        cel.BoolType)
    program, _ := env.Program(ast)

    emptyClaims := map[string]string{}
    eval(program, request(auth("other:me@acme.co", emptyClaims), time.Now()))
}
```

執行程式碼並指定時間

重新執行程式，

```
go run .
```

您應該會看到下列錯誤訊息：

```
=== Exercise 3: Logical AND/OR ===

request.auth.claims.group == 'admin'
  || request.auth.principal == 'user:me@acme.co'

----- input -----
request = time: <
  seconds: 1588989833
>
auth: <
  principal: "user:me@acme.co"
  claims: <
    >
  >
>

----- result -----
value: true (types.Bool)

----- input -----
request = time: <
  seconds: 1588989833
>
auth: <
  principal: "other:me@acme.co"
  claims: <
    >
  >
>

----- result -----
error: no such key: group
```

在 protobuf 中，我們知道要預期哪些欄位和型別。在對應和 JSON 值中，我們不知道索引鍵是否會存在。由於缺少鍵時沒有安全預設值，CEL 會預設為錯誤。

8. 自訂函式

雖然 CEL 包含許多內建函式，但有時自訂函式會很有用。舉例來說，自訂函式可用於改善常見情況的使用者體驗，或公開與情境相關的狀態

在本練習中，我們將探討如何公開函式，將常用的檢查項目封裝在一起。

呼叫自訂函式

首先，請建立程式碼來設定名為 `contains` 的覆寫，判斷地圖中是否存在鍵，以及該鍵是否具有特定值。為函式定義和函式繫結保留預留位置：

```
// exercise4 demonstrates how to extend CEL with custom functions.
// Declare a `contains` member function on map types that returns a boolean
// indicating whether the map contains the key-value pair.
func exercise4() {
    fmt.Println("=== Exercise 4: Customization ===\n")
    // Determine whether an optional claim is set to the proper value. The
    // custom map.contains(key, value) function is used as an alternative to:
    //   key in map && map[key] == value
    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        // Declare the request.
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
        // Add the custom function declaration and binding here with cel.Function()
    )
    ast := compile(env,
        `request.auth.claims.contains('group', 'admin')`,
        cel.BoolType)

    // Construct the program plan and provide the 'contains' function impl.
    // Output: false
    program, _ := env.Program(ast)
    emptyClaims := map[string]string{}
    eval(
        program,
        request(auth("user:me@acme.co", emptyClaims), time.Now()),
    )
    fmt.Println()
}
```

執行程式碼並瞭解錯誤

重新執行程式碼，您應該會看到下列錯誤：

```
ERROR: <input>:1:29: found no matching overload for 'contains' applied to 'map(string, dyn).
(string, string)'
| request.auth.claims.contains('group', 'admin')
| .....^
```

如要修正錯誤，我們需要在目前宣告要求變數的宣告清單中，加入 `contains` 函式。

附註：宣告函式與宣告變數類似。函式必須指出其通用名稱，並列舉一組具有不重複簽章的過載。

新增下列 3 行，宣告參數化型別。(這與 CEL 的任何函式多載一樣複雜)

```
// exercise4 demonstrates how to extend CEL with custom functions.
// Declare a `contains` member function on map types that returns a boolean
// indicating whether the map contains the key-value pair.
func exercise4() {
    fmt.Println("=== Exercise 4: Customization ===\n")
    // Determine whether an optional claim is set to the proper value. The custom
    // map.contains(key, value) function is used as an alternative to:
    //   key in map && map[key] == value

    // Useful components of the type-signature for 'contains'.
    typeParamA := cel.TypeParamType("A")
    typeParamB := cel.TypeParamType("B")
    mapAB := cel.MapType(typeParamA, typeParamB)

    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        // Declare the request.
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
        // Add the custom function declaration and binding here with cel.Function()
    )
    ast := compile(env,
        `request.auth.claims.contains('group', 'admin')`,
        cel.BoolType)

    // Construct the program plan.
    // Output: false
    program, _ := env.Program(ast)
    emptyClaims := map[string]string{}
    eval(program, request(auth("user:me@acme.co", emptyClaims), time.Now()))
    fmt.Println()
}
```

新增自訂函式

接著，我們會新增 `contains` 函式，該函式會使用參數化型別：


```
// exercise4 demonstrates how to extend CEL with custom functions.
// Declare a `contains` member function on map types that returns a boolean
// indicating whether the map contains the key-value pair.
func exercise4() {
    fmt.Println("=== Exercise 4: Customization ===\n")
    // Determine whether an optional claim is set to the proper value. The custom
    // map.contains(key, value) function is used as an alternative to:
    //   key in map && map[key] == value

    // Useful components of the type-signature for 'contains'.
    typeParamA := cel.TypeParamType("A")
    typeParamB := cel.TypeParamType("B")
    mapAB := cel.MapType(typeParamA, typeParamB)

    // Env declaration.
    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        // Declare the request.
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
        // Declare the custom contains function and its implementation.
        cel.Function("contains",
            cel.MemberOverload(
                "map_contains_key_value",
                []*cel.Type{mapAB, typeParamA, typeParamB},
                cel.BoolType,
                // Provide the implementation using cel.FunctionBinding()
            ),
        ),
    )
    ast := compile(env,
        `request.auth.claims.contains('group', 'admin')`,
        cel.BoolType)

    // Construct the program plan and provide the 'contains' function impl.
    // Output: false
    program, _ := env.Program(ast)
    emptyClaims := map[string]string{}
    eval(program, request(auth("user:me@acme.co", emptyClaims), time.Now()))
    fmt.Println()
}
```

執行程式來瞭解錯誤

執行練習。您應該會看到下列錯誤訊息，指出缺少執行階段函式：

```
----- result -----
error: no such overload: contains
```

使用 `cel.FunctionBinding()` 函式，為 `NewEnv` 宣告提供函式實作：

```
// exercise4 demonstrates how to extend CEL with custom functions.
//
// Declare a contains member function on map types that returns a boolean
// indicating whether the map contains the key-value pair.
func exercise4() {
    fmt.Println("=== Exercise 4: Customization ===\n")
    // Determine whether an optional claim is set to the proper value. The custom
    // map.contains(key, value) function is used as an alternative to:
    //   key in map && map[key] == value

    // Useful components of the type-signature for 'contains'.
    typeParamA := cel.TypeParamType("A")
    typeParamB := cel.TypeParamType("B")
    mapAB := cel.MapType(typeParamA, typeParamB)

    // Env declaration.
    env, _ := cel.NewEnv(
        cel.Types(&rpcpb.AttributeContext_Request{}),
        // Declare the request.
        cel.Variable("request",
            cel.ObjectType("google.rpc.context.AttributeContext.Request"),
        ),
        // Declare the custom contains function and its implementation.
        cel.Function("contains",
            cel.MemberOverload(
                "map_contains_key_value",
                []*cel.Type{mapAB, typeParamA, typeParamB},
                cel.BoolType,
                cel.FunctionBinding(mapContainsKeyValue)),
            ),
        ),
    ast := compile(env,
        `request.auth.claims.contains('group', 'admin')`,
        cel.BoolType)

    // Construct the program plan.
    // Output: false
    program, err := env.Program(ast)
    if err != nil {
        glog.Exit(err)
    }

    eval(program, request(auth("user:me@acme.co", emptyClaims), time.Now()))
    claims := map[string]string{"group": "admin"}
    eval(program, request(auth("user:me@acme.co", claims), time.Now()))
    fmt.Println()
}
```

程式現在應可順利執行：

```
=== Exercise 4: Custom Functions ===
```

```
request.auth.claims.contains('group', 'admin')
```

```
----- input -----
```

```
request = time: <
```

```
  seconds: 1569302377
```

```
>
```

```
auth: <
```

```
  principal: "user:me@acme.co"
```

```
  claims: <
```

```
>
```

```
>
```

```
----- result -----
```

```
value: false (types.Bool)
```

如果已提出著作權聲明，會發生什麼情況？

如要獲得額外學分，請嘗試在輸入內容中設定管理員憑證，驗證當憑證存在時，contains 溢位也會傳回 true。您應該會看到以下的輸出內容：

```
=== Exercise 4: Customization ===
```

```
request.auth.claims.contains('group','admin')
```

```
----- input -----
```

```
request = time: <  
  seconds: 1588991010
```

```
>
```

```
auth: <  
  principal: "user:me@acme.co"  
  claims: <  
    >
```

```
>
```

```
----- result -----
```

```
value: false (types.Bool)
```

```
----- input -----
```

```
request = time: <  
  seconds: 1588991010
```

```
>
```

```
auth: <  
  principal: "user:me@acme.co"  
  claims: <  
    fields: <  
      key: "group"  
      value: <  
        string_value: "admin"
```

```
>
```

```
>
```

```
>
```

```
>
```

```
----- result -----
```

```
value: true (types.Bool)
```

最佳做法：根據過載 ID 的類型和函式名稱宣告過載 ID，例如 `targetType_func_argType_argType`。如果 `argType` 是型別參數，請使用描述性名稱，不要使用簡單型別名稱。

繼續之前，請先檢查 `mapContainsKeyValue` 函式本身：

```
// mapContainsKeyValue implements the custom function:
// map.contains(key, value) -> bool.
func mapContainsKeyValue(args ...ref.Val) ref.Val {
    // The declaration of the function ensures that only arguments which match
    // the mapContainsKey signature will be provided to the function.
    m := args[0].(traits.Mapper)

    // CEL has many interfaces for dealing with different type abstractions.
    // The traits.Mapper interface unifies field presence testing on proto
    // messages and maps.
    key := args[1]
    v, found := m.Find(key)

    // If not found and the value was non-nil, the value is an error per the
    // `Find` contract. Propagate it accordingly. Such an error might occur with
    // a map whose key-type is listed as 'dyn'.
    if !found {
        if v != nil {
            return types.ValOrErr(v, "unsupported key type")
        }
        // Return CEL False if the key was not found.
        return types.False
    }
    // Otherwise whether the value at the key equals the value provided.
    return v.Equal(args[2])
}
```

為盡可能簡化擴充作業，自訂函式的簽章會預期 `ref.Val` 類型的引數。這裡的取捨是，擴充的便利性會增加實作者的負擔，必須確保所有值型別都經過適當處理。如果輸入引數類型或數量與函式宣告不符，應傳回 `no such overload` 錯誤。

`cel.FunctionBinding()` 會新增執行階段型別防護，確保執行階段合約與環境中經過型別檢查的宣告相符。

最佳做法：當輸入內容與預期的執行階段合約不符時，請使用 `types.ValOrErr` 或 `types.MaybeNoSuchOverload` 建構及傳播錯誤。

9. 建構 JSON

CEL 也可以產生非布林值的輸出內容，例如 JSON。在函式中新增下列內容：

```
// exercise5 covers how to build complex objects as CEL literals.
//
// Given the input now, construct a JWT with an expiry of 5 minutes.
func exercise5() {
    fmt.Println("=== Exercise 5: Building JSON ===\n")
    env, _ := cel.NewEnv(
        // Declare the 'now' variable as a Timestamp.
        // cel.Variable("now", cel.TimestampType),
    )
    // Note the quoted keys in the CEL map literal. For proto messages the
    // field names are unquoted as they represent well-defined identifiers.
    ast := compile(env, `
        {'sub': 'serviceAccount:delegate@acme.co',
         'aud': 'my-project',
         'iss': 'auth.acme.com:12350',
         'iat': now,
         'nbf': now,
         'exp': now + duration('300s'),
         'extra_claims': {
             'group': 'admin'
         }}`,
        cel.MapType(cel.StringType, cel.DynType))

    program, _ := env.Program(ast)
    out, _, _ := eval(
        program,
        map[string]interface{}{
            "now": &tpb.Timestamp{Seconds: time.Now().Unix()},
        },
    )
    fmt.Printf("----- type conversion -----\n%v\n", out)
    fmt.Println()
}
```

執行程式碼

重新執行程式碼，您應該會看到下列錯誤：

```
ERROR: <input>:5:11: undeclared reference to 'now' (in container '')
|   'iat': now,
|   .....^
... and more ...
```

將 `now` 變數的 `cel.TimestampType` 類型宣告新增至 `cel.NewEnv()`，然後再次執行：

```

// exercise5 covers how to build complex objects as CEL literals.
//
// Given the input now, construct a JWT with an expiry of 5 minutes.
func exercise5() {
    fmt.Println("=== Exercise 5: Building JSON ===\n")
    env, _ := cel.NewEnv(
        cel.Variable("now", cel.TimestampType),
    )
    // Note the quoted keys in the CEL map literal. For proto messages the
    // field names are unquoted as they represent well-defined identifiers.
    ast := compile(env, `
        {'sub': 'serviceAccount:delegate@acme.co',
         'aud': 'my-project',
         'iss': 'auth.acme.com:12350',
         'iat': now,
         'nbf': now,
         'exp': now + duration('300s'),
         'extra_claims': {
             'group': 'admin'
         }}`,
        cel.MapType(cel.StringType, cel.DynType))

    // Hint:
    // Convert `out` to JSON using the valueToJSON() helper method.
    // The valueToJSON() function calls ConvertToNative(&structpb.Value{})
    // to adapt the CEL value to a protobuf JSON representation and then
    // uses the jsonpb.Marshaler utilities on the output to render the JSON
    // string.
    program, _ := env.Program(ast)
    out, _, _ := eval(
        program,
        map[string]interface{}{
            "now": time.Now(),
        },
    )
    fmt.Printf("----- type conversion -----\n%v\n", out)
    fmt.Println()
}

```

重新執行程式碼，應該就會成功：

```
=== Exercise 5: Building JSON ===
```

```
{'aud': 'my-project',
 'exp': now + duration('300s'),
 'extra_claims': {
   'group': 'admin'
 },
 'iat': now,
 'iss': 'auth.acme.com:12350',
 'nbf': now,
 'sub': 'serviceAccount:delegate@acme.co'
}
```

```
----- input -----
```

```
now = seconds: 1569302377
```

```
----- result -----
```

```
value: &{0xc0002eaf00 map[aud:my-project exp:{0xc0000973c0} extra_claims:0xc000097400 iat:
{0xc000097300} iss:auth.acme.com:12350 nbf:{0xc000097300}
sub:serviceAccount:delegate@acme.co] {0x8c8f60 0xc00040a6c0 21}} (*types.baseMap)
```

```
----- type conversion -----
```

```
&{0xc000313510 map[aud:my-project exp:{0xc000442510} extra_claims:0xc0004acdc0 iat:
{0xc000442450} iss:auth.acme.com:12350 nbf:{0xc000442450}
sub:serviceAccount:delegate@acme.co] {0x9d0ce0 0xc0004424b0 21}}
```

注意：時間戳記和持續時間是已知的類型，在 CEL 中有特殊的便利類型。 `google.protobuf.Timestamp` 和 `google.protobuf.Duration` 訊息類型等同於便利類型，但所有已知類型的欄位都無法直接存取，而是由成員函式調解。

程式會執行，但輸出值 `out` 需要明確轉換為 JSON。在這種情況下，內部 CEL 表示法可轉換為 JSON，因為它只會參照 JSON 可支援的型別，或是具有已知的 [Proto 至 JSON 對應](#)。

```
// exercise5 covers how to build complex objects as CEL literals.
//
// Given the input now, construct a JWT with an expiry of 5 minutes.
func exercise5() {
    fmt.Println("=== Exercise 5: Building JSON ===\n")
    ...
    fmt.Printf("----- type conversion ----- \n%v\n", valueToJSON(out))
    fmt.Println()
}
```

使用 `codelab.go` 檔案中的 `valueToJSON` 輔助函式轉換類型後，您應該會看到下列額外輸出內容：

----- type conversion -----

```
{
  "aud": "my-project",
  "exp": "2019-10-13T05:54:29Z",
  "extra_claims": {
    "group": "admin"
  },
  "iat": "2019-10-13T05:49:29Z",
  "iss": "auth.acme.com:12350",
  "nbf": "2019-10-13T05:49:29Z",
  "sub": "serviceAccount:delegate@acme.co"
}
```

步驟 10 · 約 5 分鐘

10. 建構 Proto

CEL 可以為編譯到應用程式中的任何訊息類型建構 protobuf 訊息。新增函式，從輸入的 `jwt` 建構 `google.rpc.context.AttributeContext.Request`

```

// exercise6 describes how to build proto message types within CEL.
//
// Given an input jwt and time now construct a
// google.rpc.context.AttributeContext.Request with the time and auth
// fields populated according to the go/api-attributes specification.
func exercise6() {
    fmt.Println("=== Exercise 6: Building Protos ===\n")

    // Construct an environment and indicate that the container for all references
    // within the expression is `google.rpc.context.AttributeContext`.
    requestType := &rpcpb.AttributeContext_Request{}
    env, _ := cel.NewEnv(
        // Add cel.Container() option for 'google.rpc.context.AttributeContext'
        cel.Types(requestType),
        // Add cel.Variable() option for 'jwt' as a map(string, Dyn) type
        // and for 'now' as a timestamp.
    )

    // Compile the Request message construction expression and validate that
    // the resulting expression type matches the fully qualified message name.
    //
    // Note: the field names within the proto message types are not quoted as they
    // are well-defined names composed of valid identifier characters. Also, note
    // that when building nested proto objects, the message name needs to prefix
    // the object construction.
    ast := compile(env, `
        Request{
            auth: Auth{
                principal: jwt.iss + '/' + jwt.sub,
                audiences: [jwt.aud],
                presenter: 'azp' in jwt ? jwt.azp : "",
                claims: jwt
            },
            time: now
        }`,
        cel.ObjectType("google.rpc.context.AttributeContext.Request"),
    )
    program, _ := env.Program(ast)

    // Construct the message. The result is a ref.Val that returns a dynamic
    // proto message.
    out, _, _ := eval(
        program,
        map[string]interface{}{
            "jwt": map[string]interface{}{
                "sub": "serviceAccount:delegate@acme.co",
                "aud": "my-project",
                "iss": "auth.acme.com:12350",
                "extra_claims": map[string]string{
                    "group": "admin",
                },
            },
        },
    ),

```

```

        "now": time.Now(),
    },
)

// Hint: Unwrap the CEL value to a proto. Make sure to use the
// `ConvertToNative(reflect.TypeOf(requestType))` to convert the dynamic proto
// message to the concrete proto message type expected.
fmt.Printf("----- type unwrap -----\n%v\n", out)
fmt.Println()
}

```

執行程式碼

重新執行程式碼，您應該會看到下列錯誤：

```

ERROR: <input>:2:10: undeclared reference to 'Request' (in container '')
|   Request{
|   .....^

```

容器基本上等同於命名空間或套件，但甚至可以細到 protobuf 訊息名稱。CEL 容器會使用與 [Protobuf](#) 和 [C++](#) 相同的命名空間解析規則，判斷特定變數、函式或型別名稱的宣告位置。

假設容器為 `google.rpc.context.AttributeContext`，型別檢查器和評估工具會嘗試以下所有變數、型別和函式的 ID 名稱：

- `google.rpc.context.AttributeContext.<id>`
- `google.rpc.context.<id>`
- `google.rpc.<id>`
- `google.<id>`
- `<id>`

如果是絕對名稱，請在變數、型別或函式參照前面加上開頭點。在範例中，運算式 `.<id>` 只會搜尋頂層 `<id>` 識別碼，不會先檢查容器。

請嘗試將 `cel.Container("google.rpc.context.AttributeContext")` 選項指定至 CEL 環境，然後再次執行：

```
// exercise6 describes how to build proto message types within CEL.
//
// Given an input jwt and time now construct a
// google.rpc.context.AttributeContext.Request with the time and auth
// fields populated according to the go/api-attributes specification.
func exercise6() {
    fmt.Println("=== Exercise 6: Building Protos ===\n")
    // Construct an environment and indicate that the container for all references
    // within the expression is `google.rpc.context.AttributeContext`.
    requestType := &rpcpb.AttributeContext_Request{}
    env, _ := cel.NewEnv(
        // Add cel.Container() option for 'google.rpc.context.AttributeContext'
        cel.Container("google.rpc.context.AttributeContext.Request"),
        cel.Types(requestType),
        // Later, add cel.Variable() options for 'jwt' as a map(string, Dyn) type
        // and for 'now' as a timestamp.
        cel.Variable("now", cel.TimestampType),
        cel.Variable("jwt", cel.MapType(cel.StringType, cel.DynType)),
    )

    ...
}
```

您應該會看到以下輸出內容：

```
ERROR: <input>:4:16: undeclared reference to 'jwt' (in container
'google.rpc.context.AttributeContext')
|   principal: jwt.iss + '/' + jwt.sub,
|   .....^
```

... 還有許多其他錯誤...

接著，宣告 `jwt` 和 `now` 變數，程式應會如預期執行：

=== Exercise 6: Building Protos ===

```
Request{
  auth: Auth{
    principal: jwt.iss + '/' + jwt.sub,
    audiences: [jwt.aud],
    presenter: 'azp' in jwt ? jwt.azp : "",
    claims: jwt
  },
  time: now
}

----- input -----
jwt = {
  "aud": "my-project",
  "extra_claims": {
    "group": "admin"
  },
  "iss": "auth.acme.com:12350",
  "sub": "serviceAccount:delegate@acme.co"
}
now = seconds: 1588993027

----- result -----
value: &{0xc000485270 0xc000274180 {0xa6ee80 0xc000274180 22} 0xc0004be140 0xc0002bf700
false} (*types.protoObj)

----- type unwrap ----
&{0xc000485270 0xc000274180 {0xa6ee80 0xc000274180 22} 0xc0004be140 0xc0002bf700 false}
```

如要獲得額外學分，您也應該對訊息呼叫 `out.Value()` 來解除包裝型別，看看結果有何變化。

使用 CEL 建構 Proto 時的其他注意事項：

1. 系統不支援為 Proto 的 `oneof` 欄位設定條件式指派。
2. 往返 `google.protobuf.Any` 時發生問題

需要設定 `oneof` 時，請先測試所需值是否存在，再建構訊息，或擴充 CEL 以納入自訂物件建構函式 (例如 `wither` 方法)，或更抽象的內容。

如果 `Any` 包含包裝函式型別 (例如 `google.protobuf.IntValue`)，CEL 會根據內容自動將 `Any` 解壓縮為 `int` 或 `null_type`。如果未設定包裝函式型別，則無法將空值指派給 `Any` 欄位，且空值會保留原始意義。目前我們只發現這個往返問題，但值得注意。

11. 巨集

巨集可用於在剖析時操控 CEL 程式。巨集會比對呼叫簽章，並操控輸入呼叫及其引數，以產生新的子運算式 AST。

巨集可用於在 AST 中實作複雜邏輯，這些邏輯無法直接以 CEL 編寫。舉例來說，`has` 巨集可啟用欄位存在性測試。`exists` 和 `all` 等理解巨集會取代函式呼叫，並對輸入清單或對映進行繫結疊代。這兩種概念都無法在語法層級實現，但可以透過巨集擴充功能實現。

新增並執行下一個練習：

```
// exercise7 introduces macros for dealing with repeated fields and maps.
//
// Determine whether the jwt.extra_claims has at least one key that starts
// with the group prefix, and ensure that all group-like keys have list
// values containing only strings that end with '@acme.co`.
func exercise7() {
    fmt.Println("=== Exercise 7: Macros ===\n")
    env, _ := cel.NewEnv(
        cel.ClearMacros(),
        cel.Variable("jwt", cel.MapType(cel.StringType, cel.DynType)),
    )
    ast := compile(env,
        `jwt.extra_claims.exists(c, c.startsWith('group'))
        && jwt.extra_claims
            .filter(c, c.startsWith('group'))
            .all(c, jwt.extra_claims[c]
                .all(g, g.endsWith('@acme.co')))` ,
        cel.BoolType)
    program, _ := env.Program(ast)

    // Evaluate a complex-ish JWT with two groups that satisfy the criteria.
    // Output: true.
    eval(program,
        map[string]interface{}{
            "jwt": map[string]interface{}{
                "sub": "serviceAccount:delegate@acme.co",
                "aud": "my-project",
                "iss": "auth.acme.com:12350",
                "extra_claims": map[string][]string{
                    "group1": {"admin@acme.co", "analyst@acme.co"},
                    "labels": {"metadata", "prod", "pii"},
                    "groupN": {"forever@acme.co"},
                },
            },
        },
    )

    fmt.Println()
}
```

您應該會看到下列錯誤：

```
ERROR: <input>:1:25: undeclared reference to 'c' (in container '')
| jwt.extra_claims.exists(c, c.startsWith('group'))
| .....^
```

...等等。

發生這些錯誤的原因是巨集尚未啟用。如要啟用巨集，請移除 `cel.ClearMacros()`，然後再次執行：

```
=== Exercise 7: Macros ===

jwt.extra_claims.exists(c, c.startsWith('group'))
  && jwt.extra_claims
    .filter(c, c.startsWith('group'))
    .all(c, jwt.extra_claims[c]
      .all(g, g.endsWith('@acme.co'))))

----- input -----
jwt = {
  "aud": "my-project",
  "extra_claims": {
    "group1": [
      "admin@acme.co",
      "analyst@acme.co"
    ],
    "groupN": [
      "forever@acme.co"
    ],
    "labels": [
      "metadata",
      "prod",
      "pii"
    ]
  },
  "iss": "auth.acme.com:12350",
  "sub": "serviceAccount:delegate@acme.co"
}

----- result -----
value: true (types.Bool)
```

最佳做法：如果使用案例不需要巨集，請在建構環境時使用 `cel.ClearMacros()` 選項停用巨集。

目前支援的巨集如下：

巨集	簽名	說明
全部	<code>r.all(var, cond)</code>	測試 <code>cond</code> 對範圍 <code>r</code> 中的 <i>所有</i> <code>var</code> 是否評估為 <code>true</code> 。

存在	r.exists(var, cond)	測試 cond 對範圍 r 中的任何 var 是否評估為 true。
exists_one	r.exists_one(var, cond)	測試範圍 r 中只有一個 var 的 cond 評估結果是否為 true。
篩選	r.filter(var, cond)	如果是清單，請建立新清單，其中範圍 r 中的每個元素變數都會滿足條件 cond。如果是對應，請建立新清單，其中範圍 r 中的每個鍵變數都會滿足條件 cond。
地圖	r.map(var, expr)	建立新清單，其中範圍 r 中的每個 var 都會由 expr 轉換。
	r.map(var, cond, expr)	與雙引數對應相同，但會在轉換值之前加入條件式 cond 篩選器。
付款卡詳情	has(a.b)	測試值 a 上是否有 b：如果是對應，則為 JSON 測試定義。針對 Proto，測試非預設的原始值或一組訊息欄位。

如果範圍 r 引數是 `map` 型別，`var` 會是地圖鍵；如果是 `list` 型別值，`var` 會是清單元素值。`all`、`exists`、`exists_one`、`filter` 和 `map` 巨集會執行 AST 重寫，以執行受輸入大小限制的 for-each 疊代。

有界限的理解確保 CEL 程式不會是圖靈完備，但會根據輸入內容以超線性時間評估。請盡量不要使用這些巨集。大量使用推導式通常是個好指標，表示自訂函式可提供更優質的使用者體驗和更出色的效能。

12. 調整

目前只有 CEL-Go 支援少數功能，但這也代表其他 CEL 實作方式的未來計畫。以下練習會展示同一項 AST 的不同計畫：

```
// exercise8 covers features of CEL-Go which can be used to improve
// performance and debug evaluation behavior.
//
// Turn on the optimization, exhaustive eval, and state tracking
// ProgramOption flags to see the impact on evaluation behavior.
func exercise8() {
    fmt.Println("=== Exercise 8: Tuning ===\n")
    // Declare the x and 'y' variables as input into the expression.
    env, _ := cel.NewEnv(
        cel.Variable("x", cel.IntType),
        cel.Variable("y", cel.UintType),
    )
    ast := compile(env,
        `x in [1, 2, 3, 4, 5] && type(y) == uint`,
        cel.BoolType)

    // Try the different cel.EvalOptions flags when evaluating this AST for
    // the following use cases:
    // - cel.OptOptimize: optimize the expression performance.
    // - cel.OptExhaustiveEval: turn off short-circuiting.
    // - cel.OptTrackState: track state and compute a residual using the
    //   interpreter.PruneAst function.
    program, _ := env.Program(ast)
    eval(program, cel.NoVars())

    fmt.Println()
}
```

最佳化工具

開啟最佳化標記後，CEL 會花費額外時間預先建構清單和對應常值，並將 in 運算子等特定呼叫最佳化為真正的集合成員資格測試：

```
// Turn on optimization.
trueVars := map[string]interface{}{"x": int64(4), "y": uint64(2)}
program, _ := env.Program(ast, cel.EvalOptions(cel.OptOptimize))
// Try benchmarking with the optimization flag on and off.
eval(program, trueVars)
```

如果針對不同輸入內容多次評估同一程式，則最佳化是個不錯的選擇。不過，如果程式只會評估一次，最佳化只會增加負擔。

Exhaustive Eval

詳盡評估可深入瞭解運算式評估行為，並提供運算式評估每個步驟的觀察值，因此有助於偵錯。

```
// Turn on exhaustive eval to see what the evaluation state looks like.
// The input is structure to show a false on the first branch, and true
// on the second.
falseVars := map[string]interface{}{"x": int64(6), "y": uint64(2)}
program, _ = env.Program(ast, cel.EvalOptions(cel.OptExhaustiveEval))
eval(program, falseVars)
```

您應該會看到每個運算式 ID 的運算式評估狀態清單：

```
----- eval states -----
1: 6 (types.Int)
2: false (types.Bool)
3: &{0xc0000336b0 [1 2 3 4 5] {0x89f020 0xc000434760 151}} (*types.baseList)
4: 1 (types.Int)
5: 2 (types.Int)
6: 3 (types.Int)
7: 4 (types.Int)
8: 5 (types.Int)
9: uint (*types.Type)
10: 2 (types.Uint)
11: true (types.Bool)
12: uint (*types.Type)
13: false (types.Bool)
```

運算式 ID 2 對應至第一個分支中的 `in` 運算子結果，運算式 ID 11 則對應至第二個分支中的 `==` 運算子。在正常評估下，運算式會在計算 2 之後短路。如果 `y` 不是 `uint`，狀態就會顯示運算式失敗的兩個原因，而不只是一個。

注意：在詳盡評估期間，系統尚未正確追蹤理解狀態。請參閱 [issues/121](#)。

步驟 13 · 約 5 分鐘

13. 涵蓋內容

如需運算式引擎，建議使用 CEL。如果專案需要執行使用者設定，且效能至關重要，CEL 就是理想的選擇。

希望您在先前的練習中，已熟悉將資料傳遞至 CEL，並取得輸出內容或決策。

我們希望您已瞭解可執行的作業類型，包括布林決策，以及產生 JSON 和 Protobuffer 訊息。

希望您已瞭解如何使用運算式，以及運算式的作用。我們也瞭解常見的延長方式。
